The Sovereign Garden: A Protocol for Agentic, Decentralized Thought

1. Executive Summary: The Architecture of Digital Memory

The contemporary digital landscape is characterized by a "stream" paradigm—a relentless, ephemeral flow of information that privileges recency over relevance and engagement over understanding. This report outlines the architectural, philosophical, and technical specifications for a new "Vibecoding" project: a local-first, AI-enabled digital garden built on the Nostr protocol. This project seeks to invert the stream, establishing a "garden" paradigm where knowledge is cultivated, interlinked, and preserved as evergreen assets.[1]

The proposed application represents a convergence of three distinct technological movements:

1. **The Renaissance of Tools for Thought:** Leveraging the Zettelkasten method and knowledge graph theory to facilitate deep, structural cognition.[1]
2. **The Local-First Movement:** Prioritizing user sovereignty, offline capability, and data ownership through on-device storage (PGlite/WASM).[4]
3. **The Agentic Web:** Integrating browser-based Artificial Intelligence (Small Language Models) to act as proactive "gardeners" that maintain, prune, and connect the user's knowledge base.[6]

While inspired by the "Noosphere" protocol's concepts of "Spheres" and "Petnames" [8], this project strategically diverges by building upon the **Nostr** protocol.[10] This decision is driven by the requirement for "survivability"—Nostr's radical simplicity and relay-based architecture offer a censorship-resistant substrate that ensures the longevity of the user's externalized memory.[12]

This document serves as the comprehensive project constitution, technical roadmap, and agent specification. It details the design of a custom protocol layered atop Nostr, utilizing specific event kinds (NIP-54, NIP-01) and semantic linking strategies to create a decentralized, hyper-personalized memory layer for humans and their AI companions.

---

2. Project Constitution: The Garden Protocol

This section defines the philosophical core of the project, serving as the project-constitution.md requested. It establishes the "why" before the "how."

2.1 Preamble: The Right to Memory

In an era of algorithmic curation, the capacity to remember—and to organize that memory on

one's own terms—is a fundamental digital right. We assert that the "Digital Garden" is not merely a software category but a cognitive sanctuary. It is a space where thoughts are treated as durable assets rather than disposable content. This project, "The Garden," functions as a protocol for reclaiming the digital mind, adhering to the "Vibecoding" ethos: software that feels biological, responsive, and deeply personal.[9]

2.2 Article I: The Principle of Sovereignty

The architecture must strictly adhere to a **Local-First** topology. The "Source of Truth" is the user's device, not a server, not a blockchain, and not a relay.

- **Data Ownership:** Users cryptographically sign every thought (Note) using their Nostr private key (NIP-01). This ensures that even if the application developers disappear, the user retains full custody of their intellectual labor.[12]
- **Credible Exit:** As emphasized in the Noosphere principles, users must possess the ability to "exit" the system without data loss. The data format must be open (JSON/Markdown), and the synchronization protocol (Nostr) must allow for effortless migration between relays.[13]
- **Survivability:** The system is designed for the "Long Now." Dependencies on centralized APIs or proprietary cloud services are rejected in favor of open protocols and local computation (WASM).[4]

2.3 Article II: The Agentic Contract

We are transitioning from passive "Tools for Thought" to active "Agents for Thought." However, this agency comes with strict boundaries.

- **Augmentation, Not Replacement:** The AI Agent (The Gardener) exists to serve the user's intent. It suggests connections, surfaces forgotten notes, and synthesizes contradictions, but it never modifies the "canonical" text without explicit user confirmation.[15]
- **Privacy by Design:** "Agentic" does not mean "Cloud-based." The AI models run locally in the browser (via WebLLM/Transformers.js). No user data is transmitted to third-party inference APIs (e.g., OpenAI) unless the user explicitly configures a "proxy" mode for heavier tasks.[7]
- **Transparency:** The agent's reasoning must be interpretable. When the Gardener suggests a link between Note A and Note B, it must provide a semantic rationale (e.g., "Both notes discuss 'Emergent Complexity' in biological systems").[6]

2.4 Article III: The Noospheric Topology

We adopt the structural metaphors of the Noosphere protocol but implement them on Nostr:

- **Spheres:** A user's collection of notes is their "Sphere," cryptographically bound to their Public Key (Pubkey).
- **Petnames:** We reject global namespaces (DNS) in favor of "Petnames"—local, human-readable names assigned by the user to other Spheres they trust (e.g., "Alice" instead of npub1...).[13] This creates a "Web of Trust" that filters noise and prioritizes

signal.
- **Public Learning, Private Cultivation:** The default state of a note is private (encrypted or local-only). Publishing is a deliberate act of opening the garden gate to the broader Noosphere (the Nostr network).[9]

---

3. The Theoretical Framework: From Zettelkasten to Agentic Graph

To build a "hyper-personalized memory layer," we must operationalize the Zettelkasten method into a rigorous data model that AI agents can parse and reason about.

3.1 The Atomicity Requirement

The Zettelkasten method's core tenet is **atomicity**: one idea per note.[1]

- **Human Cognitive Benefit:** Forces the user to clarify their thinking and deconstruct complex arguments into constituent premises.
- **AI Technical Benefit:** Atomicity is critical for Retrieval-Augmented Generation (RAG). When a user asks an AI agent a question, the system searches for relevant context. If notes are long and rambling (e.g., a "Daily Journal" entry covering five different topics), the Vector Search retrieves irrelevant noise, hallucinating the answer. By enforcing atomic notes, we create "clean vectors"—dense numerical representations of single concepts—that allow the AI to retrieve and compose answers with high precision.[2]

3.2 The Semantic Knowledge Graph

Traditional Zettelkasten relies on "direct links" (Note A links to Note B). For an agentic system, this is insufficient. We require **Semantic Triples** (Subject - Predicate - Object).[19]

- **Standard Link:**] (Implies "related to").
- **Semantic Link:**] | type:refutes or] | type:supports.
- **Graph Utility:** This allows the AI agent to perform "Multi-hop Reasoning." For example: "Note A *supports* Note B, and Note B *refutes* Note C; therefore, Note A indirectly challenges Note C." This logic is impossible with standard Wiki-style linking.[2]

3.3 The Digital Garden as an Agent Environment

The application is not merely a database; it is a persistent environment for the AI. The agent "lives" in the background loop of the application.[6]

- **The Sleeping State:** When the user is idle, the agent "dreams" (indexes new notes, recalculates vector embeddings, and looks for "orphan" notes that lack connections).[6]
- **The Waking State:** When the user writes, the agent acts as a "co-pilot," analyzing the real-time buffer to suggest relevant past notes or warn of contradictions with established beliefs.[15]

---

4. Architectural Specification: The "Vibecoding" Stack

The technical implementation prioritizes the "Local-First" requirement while leveraging Nostr for distribution. This architecture is defined as "Client-Relay-Client," where the relay is a dumb pipe and the client is the smart brain.

4.1 The Database Layer: PGlite & WASM

For a "Vibecoding" project—implying high performance and modern tooling—we select **PGlite** (Postgres in WebAssembly) over traditional options like RxDB or raw IndexedDB.[4]

Comparison and Decision Matrix:

Feature	PGlite (Postgres WASM)	RxDB (NoSQL)	Raw IndexedDB
Vector Search	**Native (pgvector)**	Requires Plugin/Slow	Manual Implementation
Query Power	Full SQL	Mango Query	Key-Value only
Performance	Near-native C++ speed	JS Object overhead	Fast read/write
Sync Logic	Custom (via Nostr)	Built-in (CouchDB)	Manual
Size	~3MB (Gzipped)	Lightweight	Zero overhead

Why PGlite? The integration of pgvector is the killer feature. It allows us to store the note's text *and* its 384-dimensional vector embedding in the same table. We can run complex SQL queries that combine semantic similarity with metadata filtering (e.g., "Find notes semantically similar to 'Decentralization' created after 2024") in a single, sub-millisecond query execution.[4] This aligns with the "offline-friendly" requirement, as the vector math happens locally in WASM, not on a server.

4.2 The Network Layer: Nostr as Sync Protocol

Nostr is repurposed here from a "Twitter alternative" to a "Decentralized Sync Protocol." We utilize Nostr's **Parameterized Replaceable Events (NIP-33/Kind 3xxxx)** to handle the mutability of garden notes.[22]

The Survivability Argument: Building on Nostr offers "survivability" because the data is not locked in a proprietary server. The "backend" is any set of relays the user chooses (e.g., wss://relay.damus.io, wss://nos.lol). If the custom "VibeGarden" app ceases to exist, the user can point a generic Nostr wiki client (like Wikistr) to their relays and recover their data.[23]

4.3 The Storage Layer: Blossom (NIP-96)

Nostr events are small (limited to ~64KB). They cannot store images or heavy attachments. We integrate **Blossom** (Blob Storage on Nostr).[25]

- **Mechanism:** When a user uploads an image, the client signs an authorization header (NIP-98), uploads the file to a Blossom server, and receives a SHA-256 hash.
- **Linkage:** The image is referenced in the note via markdown

![alt](https://blossom.server/hash.jpg) and a corresponding **Kind 1063 (File Metadata)** event is published to link the hash to the user's pubkey.[27]

---

5. Protocol Specification: The Garden on Nostr

We define a specific "Garden Protocol" layered on top of Nostr primitives. This ensures interoperability with existing clients (like Wikistr) while adding the custom metadata needed for our Agentic features.

5.1 Event Kinds Registry

We will utilize the following Nostr Event Kinds. Where standard kinds exist (NIP-54), we align with them. Where new functionality is needed, we define custom range kinds or use specific tags.

Kind	Name	Specification	Usage in VibeGarden
30818	**Wiki Article**	**NIP-54**	The core "Note" unit. Addressable via d tag (slug).
1	**Short Note**	**NIP-01**	Fleeting thoughts, "Daily Log," or "Inbox" items.
30001	**Semantic Edge**	*Custom*	Defines a typed link between two notes (Subject, Predicate, Object).
1063	**File Metadata**	**NIP-96**	Metadata for images/attachments stored on Blossom.
10002	**Relay List**	**NIP-65**	Defines where the user's garden is hosted.
3	**Contact List**	**NIP-02**	The "Petname" registry (Who the user follows).

5.2 The Note Data Model (Kind 30818)

We strictly adhere to NIP-54 (Wiki) to ensure the notes are readable by other clients.[22] However, we extend the tags to include agentic metadata.

Event Structure:

JSON

```
{
  "kind": 30818,
  "pubkey": "user_public_key",
  "created_at": 1706227200,
  "tags": ["d", "concept-digital-garden"], // The URL slug (Deterministic ID)
   ,
    ["summary", "A philosophy of maintaining evergreen notes..."],
    ["t", "pkm"], // Hashtag for discovery
    ["t", "software-architecture"],
    // Standard NIP-54 Link (Wiki Link)
    ["e", "event_id_of_zettelkasten", "wss://relay.damus.io", "mention"],
    // PROPOSAL: Agentic Embedding Tag (Optional - for sync, or kept local)
    ["client", "vibegarden"],
    // Language
  ],
  "content": "# The Digital Garden\n\n This is the body of the note using]..."
}
```

5.3 The Semantic Edge (Kind 30001 - Custom)

While NIP-54 allows linking via e tags in the note, it does not support "Typed Links" (e.g., "Supports", "Refutes") natively in a way that is easily queryable without parsing Markdown. To build a robust Knowledge Graph, we introduce a **Semantic Edge Event**.

- **Rationale:** By separating edges from content, we allow the graph to evolve without editing the original note. This is crucial for "backlinking" where Note B links to Note A, but we don't want to edit Note A to reflect this.
- **Structure:**
  JSON
  ```
  {
    "kind": 30001,
    "tags": ["d", "edge-uuid-v4"],
      ["s", "slug-of-source-note"], // Subject
      ["p", "refutes"],            // Predicate (Relationship Type)
      ["o", "slug-of-target-note"], // Object
      ["w", "0.85"]                // Weight (e.g., Agent confidence score)
    ],
    "content": "Optional explanation of why this link exists."
  }
  ```

5.4 Mapping "Noosphere" to Nostr

The user requested inspiration from Noosphere (explainer.md ideas).[8] Here is the rigorous

translation of those concepts to the Nostr environment:

1. The "Sphere" → The Nostr Pubkey

In Noosphere, a "Sphere" is a data structure of a user's content. In Nostr, the **Public Key** acts as the root identity. All events signed by this key constitute the "Sphere." The kind: 10002 Relay List defines *where* this Sphere can be found physically (on which relays).

2. "Petnames" → NIP-02 Contact List

Noosphere uses local names ("@bob") to refer to public keys. We implement this using **NIP-02 (Contact List)**.

- The user adds a Pubkey to their contact list with a "petname" alias.
- **Agentic Resolution:** When the user types [[@Alice/concept-of-time]] in the editor, the Agent resolves @Alice by looking up the NIP-02 list, finding the Pubkey, and then querying relays for the Kind 30818 event with d: concept-of-time authored by that Pubkey.
- This achieves the "Hyper-local, Decentralized DNS" vision of Noosphere without a blockchain.[13]

3. "Credible Exit" → Key Export & JSON Since Nostr events are standardized JSON, "Exit" is trivial. The user can simply export their database to a .jsonl file. Any other Nostr-compatible wiki viewer can import this, verifying the signatures to ensure integrity.[14]

---

6. The Gardener: Agentic Architecture Specification

This section details the AI implementation, serving as the agent.md requested.

6.1 The "Gardener" Persona

The Gardener is not a chatbot. It is a **proactive background process** that observes the Garden and tends to it. It runs entirely in the browser using WebAssembly.

6.2 Technical Stack: In-Browser AI

To maintain the "Offline-Friendly" and "Privacy" requirements, we cannot rely on server-side APIs.

- **Inference Engine: WebLLM** (based on Apache TVM). It allows running quantized Large Language Models (like Llama-3-8B-Quantized or Phi-3) directly in the browser using WebGPU.[28]
- **Embedding Engine: Transformers.js** (based on ONNX Runtime). We use a lightweight model like Xenova/all-MiniLM-L6-v2 (approx 23MB) to generate 384-dimensional vectors for every note.[29]

6.3 The Agent Lifecycle

Phase 1: Ingestion & Vectorization

1. **Trigger:** User saves a note (Kind 30818).
2. **Process:**
    - The Gardener intercepts the save.

    - It strips Markdown formatting to extract raw text.
    - It passes the text to Transformers.js to generate the Embedding Vector.
    - **PGlite Action:** INSERT INTO embeddings (note_id, vector) VALUES (...).
    - **Latency Masking:** This happens in a Web Worker to avoid blocking the UI thread ("Vibecoding" smoothness).

Phase 2: The Dreaming (Background Maintenance)

1. **Trigger:** User is idle for >60 seconds.
2. **Process:**
    - The Gardener scans for **Orphan Notes** (Notes with 0 edges).
    - It performs a Vector Search to find candidates for connection.
    - **Logic:** SELECT * FROM notes ORDER BY vector <=> target_vector LIMIT 5.
    - **Filtering:** It filters out notes that are already linked.
    - **Output:** It creates a "Gardener's Suggestion" notification: "Note A seems related to Note B. Link them?".[18]

Phase 3: The Co-Pilot (Real-time RAG)

1. **Trigger:** User types ?? or opens the "Chat" pane.
2. **Process:**
    - **Retrieval:** The Agent converts the user's query into a vector.
    - **Search:** Queries PGlite for the top-k (e.g., 10) most relevant text chunks.
    - **Context Window:** It constructs a prompt: "You are a helpful gardener. Answer the user's question using ONLY the following context from their notes: [Note A]..."
    - **Generation:** WebLLM generates the response token-by-token.
3. **Vibe Check:** This interaction allows the user to "converse" with their own externalized memory.

---

7. Implementation Plan: The "Vibecoding" Roadmap

This section provides the detailed plan prompt and execution steps.

7.1 Phase I: The Seed (Local Foundation)

Objective: Build a fully functional local-first Zettelkasten with Vector Search. No networking yet.

- **Stack:** Vite + React (TypeScript), PGlite (DB), Tiptap (Editor).
- **Key Deliverable:** The db.ts module initializing PGlite with pgvector.
- **Outcome:** User can write markdown, save to OPFS, and see "Related Notes" via vector similarity.

7.2 Phase II: The Sprout (Nostr Integration)

Objective: Connect the Garden to the Noosphere.

- **Stack:** nostr-tools, ndk.
- **Key Deliverable:** The SyncManager class.

    - **Publish:** Convert local PGlite rows to Kind 30818 events and broadcast to Relays.
    - **Subscribe:** Listen for updates to the user's Pubkey and hydrate PGlite.
- **Challenge:** Handling Merge Conflicts.
    - **Strategy:** "Last Write Wins" (LWW) based on created_at timestamp (standard Nostr behavior). For a single-user garden, this is sufficient. Collaborative gardens would require CRDTs (Yjs over Nostr), but that is out of scope for MVP.[31]

7.3 Phase III: The Rhizome (Agentic Layer)

Objective: Activate the Gardener.

- **Stack:** WebLLM, Transformers.js.
- **Key Deliverable:** The AgentWorker.ts.
    - Implement the background indexing loop.
    - Build the "Chat with Garden" UI.

7.4 Detailed Plan Prompt (Kickstarter)

Copy the following prompt into your LLM to generate the initial codebase:
Role: Senior Full-Stack Architect (TypeScript/WASM focus).
Project: "VibeGarden" - A local-first, AI-enabled Digital Garden on Nostr.
Context:
We are building a "Vibecoding" application that serves as a decentralized memory layer. It combines **PGlite** (Postgres in WASM) for storage, **pgvector** for semantic search, and **Nostr** for data synchronization. The app must run entirely in the browser (Offline First).
Task:
Generate the initial architectural scaffolding code for the Core Data Layer.
Requirements:

1. **Database Service (db.ts):**
    - Initialize PGlite.
    - Install/Enable the vector extension.
    - Define a schema SQL script creating:
        - notes table: id (uuid), slug (text, unique), title, content, created_at (bigint), nostr_event_id (text).
        - embeddings table: note_id (fk), vector (vector(384)).
    - Export a singleton instance.
2. **Nostr Sync Adapter (nostr.ts):**
    - Create a function publishNote(note: Note, privateKey: string) that signs a **NIP-54 (Kind 30818)** event.
    - The d tag must be the note's slug.
    - Include a summary tag derived from the first 50 chars.
3. **Vector Service (vector.ts):**
    - Stub a function generateEmbedding(text: string): Promise<number> using a placeholder or transformers.js syntax.
    - Create a function findSimilarNotes(embedding: number) that executes a PGlite

SQL query using the <=> (cosine distance) operator.

Style Guide:

- Strict TypeScript.
- Use Functional patterns.
- Prioritize readability and "Vibe" (minimal boilerplate, clean abstractions).

Output:

Provide the file contents for package.json, src/services/db.ts, src/services/nostr.ts, and src/services/vector.ts.

---

8. Critical Analysis: Why This Approach Wins

8.1 The "Protocol within a Protocol" Strategy

By defining specific conventions (Kind 30818 + Semantic Tags) *on top* of Nostr, we create a robust data model without needing to fork the underlying protocol. This is the same strategy used by the "AT Protocol" (Bluesky), but applied to the simpler, battle-tested rails of Nostr.

8.2 The "Local-First AI" Moat

Most "AI Notes" apps (Mem.ai, Notion AI) are server-side wrappers. They break the "Privacy" and "Sovereignty" promises. By running the Vector Store (PGlite) and Inference (WebLLM) locally, "VibeGarden" offers a unique value proposition: **Intelligence without Surveillance.** This appeals strongly to the target demographic of "Tools for Thought" power users and privacy advocates.[32]

8.3 Survivability

If the VibeGarden web app is taken offline, the user's data remains safe on Nostr relays. They can build a simple script to download all kind:30818 events signed by their key, ensuring that their externalized mind is indestructible.

9. Conclusion

The "VibeGarden" project is not just an app; it is a manifesto for the future of digital memory. By weaving together the mnemonic rigor of **Zettelkasten**, the resilience of **Nostr**, and the emerging power of **Local AI**, it establishes a new paradigm: The Sovereign, Agentic Garden. This report provides the complete blueprint—philosophical, structural, and technical—to move from "Day 0" to a functional prototype.

Appendix A: Reference Tables

Table 1: Tech Stack Selection

Component	Technology	Reasoning

Frontend	React + Vite	Ecosystem speed, "Vibe" performance.
Language	TypeScript	Type safety for complex data models.
Database	**PGlite (WASM)**	Native Vector Search (pgvector) in browser.
Sync Protocol	**Nostr**	Censorship resistance, relay redundancy.
AI Inference	**WebLLM**	Runs Llama-3 locally via WebGPU.
Embeddings	**Transformers.js**	Runs BERT models locally for vectorization.
Editor	Tiptap	Headless, extensible, Markdown-friendly.

Table 2: Data Model Mapping (Noosphere vs. Nostr)

Noosphere Concept	**Nostr Implementation**
Sphere	**Public Key (Pubkey)**
Memo	**Event (Kind 30818)**
Petname	**Contact List (NIP-02/56)**
Gateway	**Relay**
CID (Content ID)	**Event ID (SHA256)**
UCAN Auth	**Schnorr Signature (NIP-01)**

Works cited

1. New to ZK: is it equivalent to knowledge graphs? : r/Zettelkasten - Reddit, accessed January 26, 2026, https://www.reddit.com/r/Zettelkasten/comments/kkl7ap/new_to_zk_is_it_equivalent_to_knowledge_graphs/
2. Zettelkasten Agentic Memory: Self-Organizing Knowledge Graph with RAG in Java - Medium, accessed January 26, 2026, https://medium.com/@visrow/zettelkasten-agentic-memory-self-organizing-knowledge-graph-with-rag-in-java-36ec2672ea57
3. How To Build Your Zettelkasten to Master AI, accessed January 26, 2026, https://zettelkasten.de/posts/how-to-build-zettelkasten-master-ai/
4. electric-sql/pglite: Embeddable Postgres with real-time, reactive bindings. - GitHub, accessed January 26, 2026, https://github.com/electric-sql/pglite
5. Show HN: PGlite – in-browser WASM Postgres with pgvector and live sync | Hacker News, accessed January 26, 2026, https://news.ycombinator.com/item?id=41224689
6. A Coding Implementation on Building Self-Organizing Zettelkasten Knowledge Graphs and Sleep-Consolidation Mechanisms - MarkTechPost, accessed January

26, 2026, https://www.marktechpost.com/2025/12/25/a-coding-implementation-on-building-self-organizing-zettelkasten-knowledge-graphs-and-sleep-consolidation-mechanisms/
7. Run Your Local LLMs as Web Agents Directly in Your Browser with BrowserOS - Reddit, accessed January 26, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1nqf3vf/run_your_local_llms_as_web_agents_directly_in/
8. Noosphere and the Global Brain - Human Energy, accessed January 26, 2026, https://www.humanenergy.io/noosphere-and-the-global-brain
9. Noosphere - Boris Mann's Homepage, accessed January 26, 2026, https://bmannconsulting.com/notes/noosphere/
10. NIPs (Nostr Improvement Proposals), accessed January 26, 2026, https://nips.nostr.com/
11. The Nostr Protocol, accessed January 26, 2026, https://nostr.how/en/the-protocol
12. Creating a Nostr Client in Typescript | by Bryson Tang | Medium, accessed January 26, 2026, https://medium.com/@brysontang/creating-a-nostr-client-in-typescript-a0ce023a0bfc
13. noosphere/design/principles.md at main - GitHub, accessed January 26, 2026, https://github.com/subconsciousnetwork/noosphere/blob/main/design/principles.md
14. Noosphere's Authority Model - PlanetMath, accessed January 26, 2026, https://planetmath.org/noospheresauthoritymodel
15. Zettelkasten AI - Saatvik - Obsidian Publish, accessed January 26, 2026, https://publish.obsidian.md/saatvik/Zettelkasten+AI
16. Show HN: A web browser agent in your Chrome side panel | Hacker News, accessed January 26, 2026, https://news.ycombinator.com/item?id=44020626
17. A Concept Model of Noosphere - The Essence of Software, accessed January 26, 2026, https://essenceofsoftware.com/drafts/noosphere-v2/
18. Unlocking Your AI's Second Brain: A Deep Dive into the Zettelkasten MCP Server, accessed January 26, 2026, https://skywork.ai/skypage/en/unlocking-ai-zettelkasten-server/1981204059986821120
19. RDF - Semantic Web Standards - W3C, accessed January 26, 2026, https://www.w3.org/RDF/
20. Semantic triple - Wikipedia, accessed January 26, 2026, https://en.wikipedia.org/wiki/Semantic_triple
21. PGlite - Run PostgreSQL Locally in Node, Bun and also in Browser : r/programming - Reddit, accessed January 26, 2026, https://www.reddit.com/r/programming/comments/1faaqz0/pglite_run_postgresql_locally_in_node_bun_and/
22. NIP54 - NIP-54 - Wiki, accessed January 26, 2026, https://nips.nostr.com/54
23. Nostr Apps, accessed January 26, 2026, https://nostrapps.com/
24. Wikistr - nostrapps.com, accessed January 26, 2026,

https://nostrapps.com/wikistr
25. Blossom Drive: Store & Retrieve Data on Public Servers Using sha256 Universal ID, accessed January 26, 2026, https://www.nobsbitcoin.com/blossom-intro/
26. hzrd149/blossom: Blobs stored simply on mediaservers - GitHub, accessed January 26, 2026, https://github.com/hzrd149/blossom
27. nostr-dev-kit/ndk-blossom - NPM, accessed January 26, 2026, https://www.npmjs.com/package/@nostr-dev-kit/ndk-blossom
28. How to run a LLM local in your web app | by Fernando Abolafio | JavaScript in Plain English, accessed January 26, 2026, https://javascript.plainenglish.io/running-llms-in-the-browser-1ac0dbd15397
29. Local JavaScript Vector Database that works offline - RxDB, accessed January 26, 2026, https://rxdb.info/articles/javascript-vector-database.html
30. I Built a Vector Store from Scratch in TypeScript - YouTube, accessed January 26, 2026, https://www.youtube.com/watch?v=lWTCykdFxLE
31. Use nostr for collaborative, decentralized, local-first applications with the Nostr-CRDT Yjs provider. - GitHub, accessed January 26, 2026, https://github.com/YousefED/nostr-crdt
32. Are AI and Zettelkasten compatible each other?, accessed January 26, 2026, https://forum.zettelkasten.de/discussion/2863/are-ai-and-zettelkasten-compatible-each-other